Présentation CCP Développeur Web & Web mobile Partie Back End

20/11/2025

Sommaire

1. Résumé du projet
2. Contexte du projet
3. Outils
4. Architecture globale
5. Méthode Merise
6. Sécurité
7. Extrait de code
8. Performances
9. Tests
10. Crates
11. Déploiement
12. Compétences couvertes
13. Conclusion

Contexte du projet : Introduction

PokeRNCP est une API REST de type Pokédex, elle permet aux utilisateurs de créer un compte et de collectionner les Pokémon des 6 premières générations.

Contexte du projet : Equipe

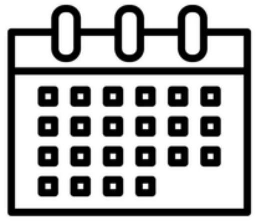
Xabi Martinez

32 ans

Holberton C#24

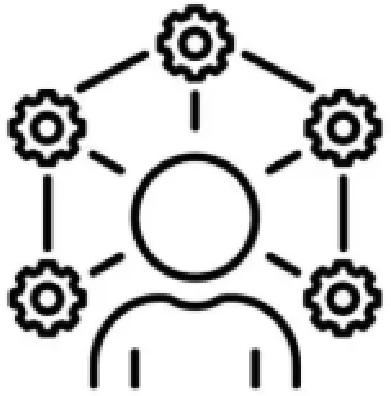
Bordeaux

Contexte du projet : Contraintes



Temps

Organisation



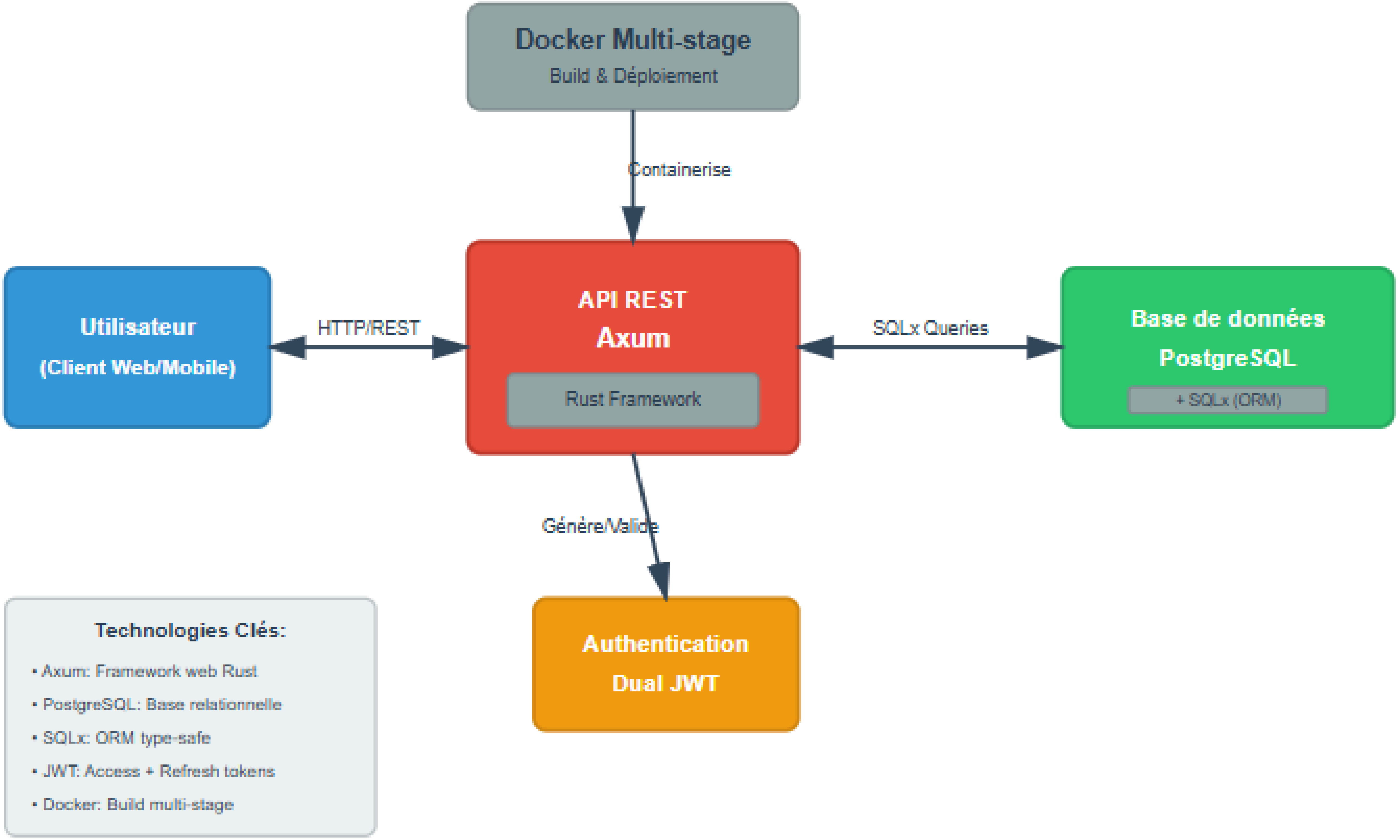
Compétences

Contexte du projet : Utilisation

- CRUD de compte
- Capture de Pokémon
- Data sur un Pokémon capturé
- Possibilité de trouver une Pokémon spécifique

Gestion du projet : Outils

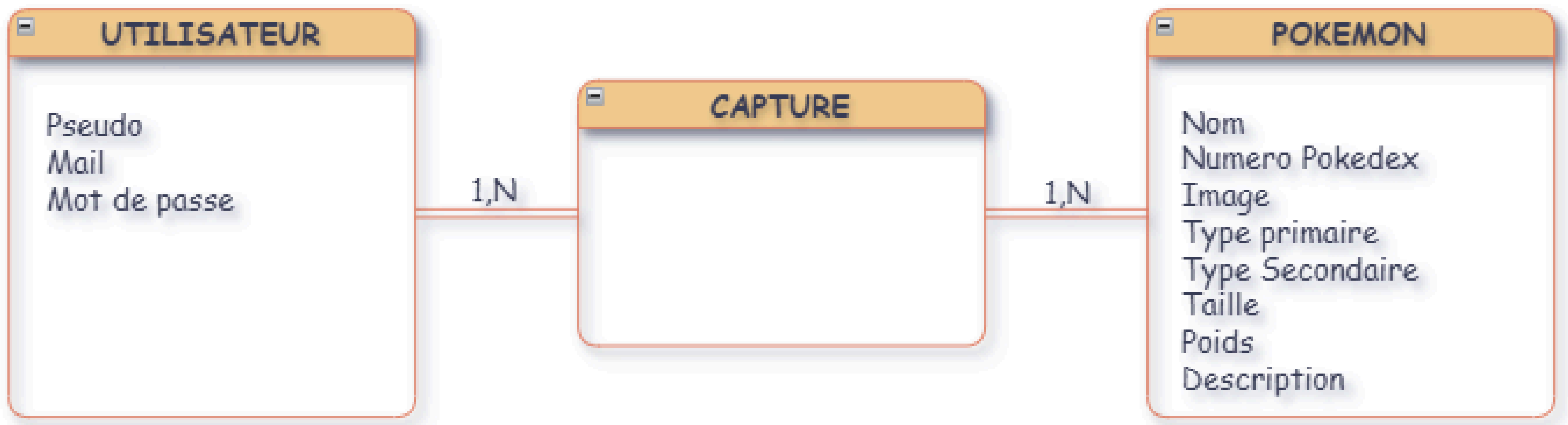
Architecture globale



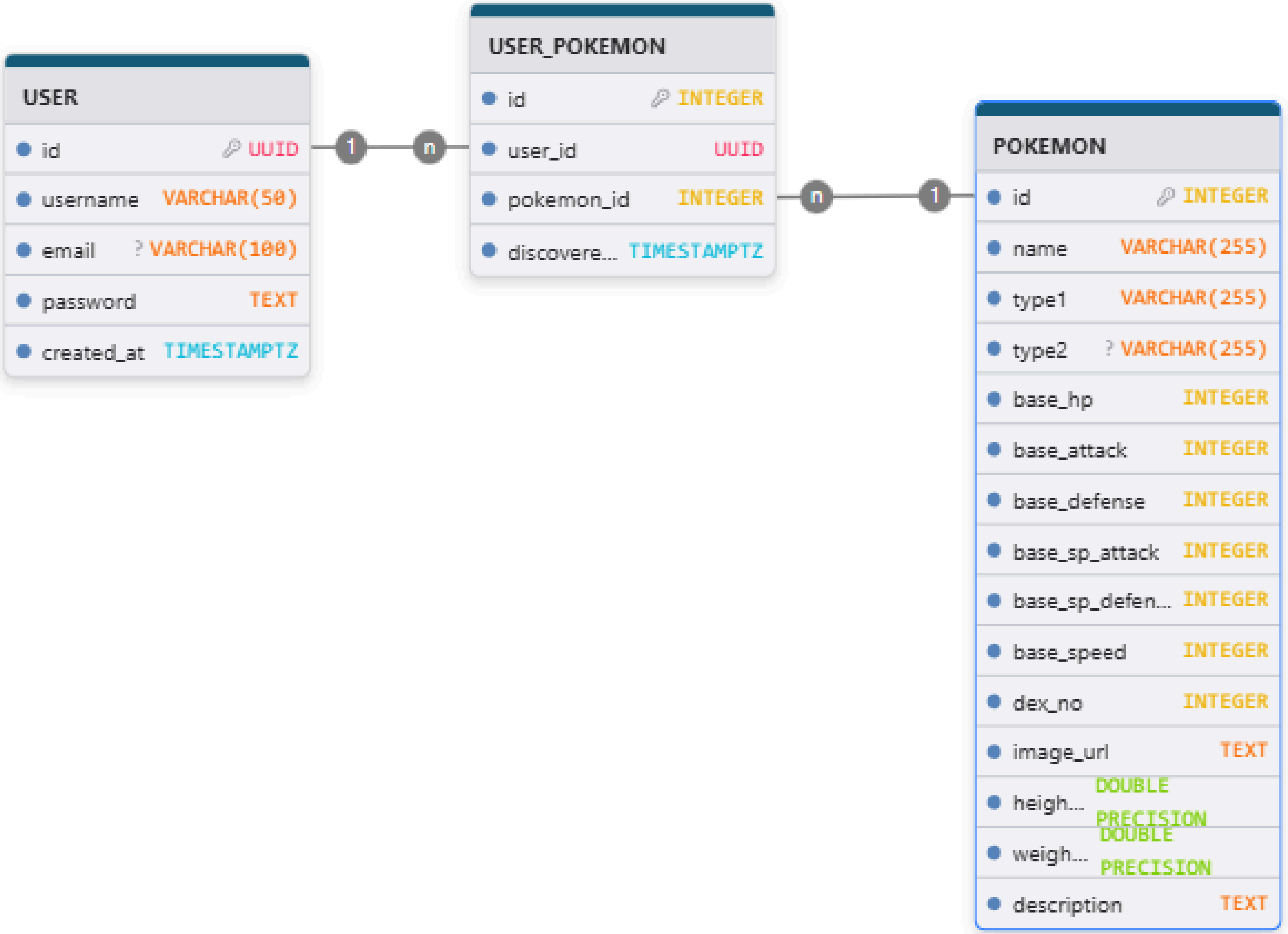
Methode Merise

- MCD : Users, Pokémon, User_Pokemon
- Relation N<=>N capturée
- Attribut dérivé caught
- Contraintes UNIQUE + cascade
- Modèle stable & normalisé

MCD


UTILISATEUR
Pseudo
Mail
Mot de passe
1,N
CAPTURE
1,N
POKEMON
Nom
Numero Pokedex
Image
Type primaire
Type Secondaire
Taille
Poids
Description

MPD


USER
id UUID
username VARCHAR(50)
email ? VARCHAR(100)
password TEXT
created_at TIMESTAMPTZ
1
n
USER_POKEMON
id INTEGER
user_id UUID
pokemon_id INTEGER
discovere... TIMESTAMPTZ
n
1
POKEMON
id INTEGER
name VARCHAR(255)
type1 VARCHAR(255)
type2 ? VARCHAR(255)
base_hp INTEGER
base_attack INTEGER
base_defense INTEGER
base_sp_attack INTEGER
base_sp_defen... INTEGER
base_speed INTEGER
dex_no INTEGER
image_url TEXT
heigh... DOUBLE PRECISION
weigh... DOUBLE PRECISION
description TEXT

Initialisation DB

init_db

```sql
-- Requis pour gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- ===== TABLE USERS =====
CREATE TABLE IF NOT EXISTS users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE,
    password TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ===== TABLE POKEMON =====
CREATE TABLE IF NOT EXISTS pokemon (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    type1 VARCHAR(20) NOT NULL,
    type2 VARCHAR(20),
    base_hp INTEGER,
    base_attack INTEGER,
    base_defense INTEGER,
    base_sp_attack INTEGER,
    base_sp_defense INTEGER,
    base_speed INTEGER
);

-- ===== TABLE ASSOCIATIVE USER_POKEMON =====
CREATE TABLE IF NOT EXISTS user_pokemon (
    id SERIAL PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    pokemon_id INTEGER NOT NULL REFERENCES pokemon(id),
    nickname VARCHAR(50),
    discovered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(user_id, pokemon_id)
);

-- ===== INDEX =====
CREATE INDEX IF NOT EXISTS idx_user_pokemon_user_id ON user_pokemon(user_id);
CREATE INDEX IF NOT EXISTS idx_user_pokemon_pokemon_id ON user_pokemon(pokemon_id);
```

Migration DB

migration

```sql
-- Migration: add_pokemon_image_and_number (UP)
-- Ajoute des colonnes pour stocker le numéro du Pokédex et l'URL de l'image

ALTER TABLE pokemon
    ADD COLUMN IF NOT EXISTS dex_no INTEGER,
    ADD COLUMN IF NOT EXISTS image_url TEXT;
```

migration

```sql
-- Migration: add_pokemon_measurements_and_weaknesses (UP)
-- Ajoute taille/poids (en unités SI) et faiblesses

ALTER TABLE pokemon
    ADD COLUMN IF NOT EXISTS height_m DOUBLE PRECISION,
    ADD COLUMN IF NOT EXISTS weight_kg DOUBLE PRECISION,
    ADD COLUMN IF NOT EXISTS weaknesses TEXT[];
```

migration

```sql
-- Drop weaknesses column and add description to pokemon
ALTER TABLE pokemon
    DROP COLUMN IF EXISTS weaknesses,
    ADD COLUMN IF NOT EXISTS description TEXT;
```

Sécurité de l’API

- JWT access court / refresh long
- Cookies HttpOnly, SameSite Lax/Strict
- Argon2 pour les mots de passe
- Middlewares Axum
- Mitigation CSRF (cookies + header)

Flux applicatifs

- Login : validation + tokens
- Refresh automatique
- Listing Pokémon
- Capture (idempotente)
- Vérifications & permissions

Extraits de code

catch function

```rust
pub async fn catch(
    CurrentUser(user_id): CurrentUser,
    State(pool): State<PgPool>,
    Json(payload): Json<CatchByNamePayload>,
) -> ApiResult<(axum::http::StatusCode, String)> {
    let pokemon_id = sqlx::query_scalar::<_, i32>(r#"SELECT id FROM pokemon WHERE name = $1"#)
        .bind(&payload.name)
        .fetch_optional(&pool)
        .await
        .map_err(to_500)?
        .ok_or_else(|| not_found("Pokémon introuvable."))?;

    let _ = sqlx::query(
        r#"
        INSERT INTO user_pokemon (user_id, pokemon_id, nickname)
        VALUES ($1, $2, $3)
        ON CONFLICT (user_id, pokemon_id) DO NOTHING
        "#,
    )
    .bind(user_id)
    .bind(pokemon_id)
    .bind(payload.nickname)
    .execute(&pool)
    .await
    .map_err(to_500)?;

    created("Pokémon marqué comme capturé.")
}
```

list_all function

```rust
pub async fn list_all(
    CurrentUser(user_id): CurrentUser,
    State(pool): State<PgPool>,
) -> ApiResult<Json<Vec<PokemonWithCaught>>> {
    let rows = sqlx::query_as::<_, PokemonWithCaught>(
        r#"
        SELECT
            p.id          AS id,
            p.name        AS name,
            p.type1       AS type1,
            p.type2       AS type2,
            p.dex_no      AS dex_no,
            p.image_url   AS image_url,
            EXISTS (
                SELECT 1 FROM user_pokemon up
                WHERE up.user_id = $1 AND up.pokemon_id = p.id
            )             AS caught
        FROM pokemon p
        ORDER BY p.id
        "#,
    )
    .bind(user_id)
    .fetch_all(&pool)
    .await
    .map_err(to_500)?;

    Ok(Json(rows))
}
```

Performance et optimisations

- Rust + Tokio
- Requêtes compilées SQLx
- Démarrage : retry DB
- Idempotence et LIMIT
- Calcul dynamique caught

Qualité et tests

- Tests d’intégration (reqwest)
- nextest : exécution rapide
- Linter clippy
- Séparation clean du code
- Documentation claire

Tests

db_test function

```rust
#[tokio::test]
async fn la_base_de_donnees_repond() {
    let pool = common::test_pool().await;
    let one: i32 = sqlx::query_scalar("SELECT 1")
        .fetch_one(pool)
        .await
        .unwrap();
    assert_eq!(one, 1);
}
```

unique_user_test function

```rust
#[tokio::test]
async fn create_user_201_et_conflit_409() {
    let (base, handle) = start_server().await;
    let client = reqwest::Client::new();
    let unique = uuid::Uuid::new_v4().to_string();
    let username = format!("user_{}", unique);
    let email = format!("{}@example.com", username);
    let res = client
        .post(format!("{}/api/users", base))
        .json(&json!({
            "username": username,
            "email": email,
            "password": "Password123!"
        }))
        .send()
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::CREATED);

    let res = client
        .post(format!("{}/api/users", base))
        .json(&json!({
            "username": username,
            "email": email,
            "password": "Password123!"
        }))
        .send()
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::CONFLICT);

    handle.abort();
    delete_user(&username).await;
}
```

Tests

```
Nextest run ID 30847ad9-3b4b-483a-a030-82761db22e59 with nextest profile: default
    Starting 17 tests across 8 binaries
        PASS [   0.709s] pokedex_rncp_backend::health_test get_root_retourne_bienvenue
        PASS [   0.710s] pokedex_rncp_backend::db_test la_base_de_donnees_repond
        PASS [   0.718s] pokedex_rncp_backend::auth logout_definit_cookies_expirants
        PASS [   0.712s] pokedex_rncp_backend::health_test path_inconnu_retourne_404
        PASS [   0.727s] pokedex_rncp_backend::pokemon list_requiert_auth
        PASS [   0.744s] pokedex_rncp_backend::auth me_requiert_authentification
        PASS [   0.741s] pokedex_rncp_backend::auth tokens_invalides_donnent_401
        PASS [   1.503s] pokedex_rncp_backend::auth me_ok_avec_bearer
        PASS [   1.542s] pokedex_rncp_backend::auth me_ok_avec_cookie_auth
        PASS [   1.554s] pokedex_rncp_backend::auth refresh_token_ok_avec_cookie_refresh
        PASS [   1.555s] pokedex_rncp_backend::auth refresh_token_requiert_bearer_et_definit_cookie
        PASS [   1.130s] pokedex_rncp_backend::pokemon list_search_catch_get
        PASS [   1.172s] pokedex_rncp_backend::user delete_user_soi_meme
        PASS [   1.604s] pokedex_rncp_backend::user create_user_201_et_conflit_409
        PASS [   1.890s] pokedex_rncp_backend::user update_user_refuse_si_different
        PASS [   2.219s] pokedex_rncp_backend::user update_user_soi_meme_modifie_username_email_et_password
        PASS [   3.182s] pokedex_rncp_backend::auth reset_mot_de_passe_retourne_token_et_confirmation_mise_a_jour
```

Crates utilisés

Tokio

A runtime for writing reliable, asynchronous, and slim applications with the Rust programming language. It is:

- **Fast**: Tokio's zero-cost abstractions give you bare-metal performance.
- **Reliable**: Tokio leverages Rust's ownership, type system, and concurrency model to reduce bugs and ensure thread safety.
- **Scalable**: Tokio has a minimal footprint, and handles backpressure and cancellation naturally.

crates.io v1.48.0 | license MIT | CI passing | chat 2k online

Website | Guides | API Docs | Chat

Overview

Tokio is an event-driven, non-blocking I/O platform for writing asynchronous applications with the Rust programming language. At a high level, it provides a few major components:

- A multithreaded, work-stealing based task scheduler.
- A reactor backed by the operating system's event queue (epoll, kqueue, IOCP, etc.).
- Asynchronous TCP and UDP sockets.

These components provide the runtime components necessary for building an asynchronous application.

RustCrypto: Argon2

crate | docs passing | argon2 no status | license Apache2.0/MIT | rustc 1.65+ | zulip join chat

Pure Rust implementation of the Argon2 password hashing function.

Documentation

About

Argon2 is a memory-hard key derivation function chosen as the winner of the Password Hashing Competition in July 2015.

It implements the following three algorithmic variants:

- **Argon2d**: maximizes resistance to GPU cracking attacks
- **Argon2i**: optimized to resist side-channel attacks
- **Argon2id**: (default) hybrid version combining both Argon2i and Argon2d

Support is provided for embedded (i.e. `no_std`) environments, including ones without `alloc` support.

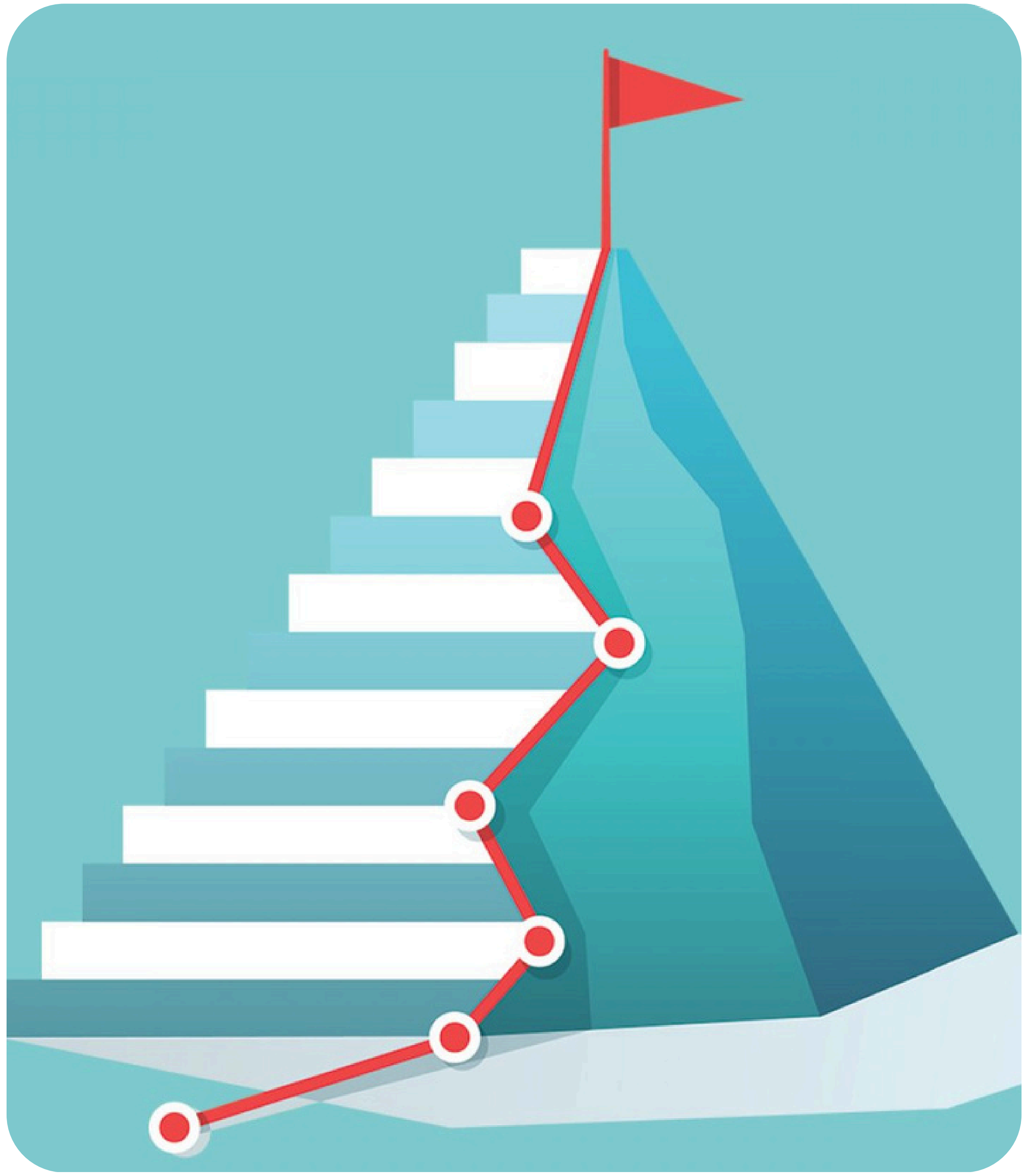
Déploiement professionnel

- Docker multi-stage Rust
- docker-compose (API + DB)
- Images légères
- Variables d’environnement
- Tunnel HTTPS possible

Compétences RNCP couvertes

- Modélisation Merise & SQL relationnel
- Développement d’API REST sécurisée
- Sécurité : hachage, JWT, cookies
- Tests, conteneurisation, déploiement

Conclusion



Travail fourni

- Système de connexion
- Système de capture dynamique
- Projet complet & opérationnel

Apprentissage

- Découverte d’Axum
- Approche professionnel
- Sécurité approfondie

Futur

- Ajout d’une CI/CD
- Ajout des features